

Moje upoznavanje sa *praktičnom* primenom optimizacije u Pythonu

Poglavlje 1: Prvi susret

Kada sam počeo da istražujem o problemima optimizacije, prvo sam se susreo sa jednim od osnovnih optimizacionih problema, jednostavno raspoređivanje resursa (Production Mix), glavna ideja: kako uz minimalne troškove ili maksimalan učinak ispuniti pravila koja nam postavlja realan sistem, ta ideja se zadržala i u drugom problemu sa kojim sam se susreo, samo na složenijem sistemu dnevnog upravljanja energijom u modernom data centru (ideja kako bi se rešio ovaj problem, naravno na dosta jednostavnijem primeru i sa izmišljenim varijablama).

1. Šta sam naučio na osnovnom primeru: Logika solvera i "Infeasibility"?

([github](#): [test_opt.py](#))

Prvi izazov bio je klasični problem proizvodnje nameštaja koristeći **Google OR-Tools (CP-SAT)**.

- **Kako funkcioniše:** Definišemo promenljive (`NewIntVar`, u određenom opsegu) kao odluke koje sistem donosi, postavimo stroga pravila, ograničenja (`model.Add`) i zadamo ciljnu funkciju (`Maximize` ili `Minimize`).
- **Glavna lekcija:** Kada sam eksperimentisao sa promenom znakova i tražio minimum uz nepromišljene uslove, solver je izbacio "Infeasible". Istražio sam šta to znači i naučio da su uslovi koje sam zadao međusobno kontradiktorni i da rešenje ne postoji.
- **CP-SAT:** Umesto da računar nasumično proba svaku kombinaciju, CP-SAT koristi fazu širenja uslova da brzo odbaci sve nemoguće opcije pre nego što krene u pretragu.

2. Prelazak na viši nivo: Matrično linearno programiranje i moj rad na sistemu za "Green Data Center"

([github](#): [green.py](#), [green_res.py](#))

Sledeći korak je bio prelazaki na realan industrijski problem: optimizacija potrošnje električne energije, rada servera i upravljanje baterijama u jednom velikom AI Data Centru kroz period od 24 sata. Ovde sam shvatio upotrebu matričnih sistema u softverskom inženjerstvu.

Prvo sam pokušao ručno da pišem ograničenja za više sati, loša odluka, zadržao sam se na tome nepotrebno dugo, amaterska greška. Kada imamo 24 sata i u svaki sat uvodimo više odluka (struja iz mreže za servere, pražnjenje baterije, punjenje baterije), broj promenljivih se znatno povećava. Ručno kucanje redova u matrici me dovodilo do dosta grešaka u indeksima koje je, bar meni, nemoguće ispratiti.

Rešenje: NumPy i SciPy

Rešenje je bilo u tome da iskoristim Python da sam generiše matricu koristeći funkcije biblioteke **NumPy**:

- **np.eye(hours)**: Shvatio sam kako se preko jedinične matrice uspostavlja "jedan na jedan" veza za svaki sat (gde sat **h** utiče samo na sopstveni energetski balans).
- **np.zeros**: Naučio sam da koristim nulte blokove da „popunim rupe“ tamo gde određena promenljiva (npr. punjenje baterije) ne učestvuje u nekom specifičnom ograničenju.
- **np.hstack** i **np.vstack**: Ovi alati su mi postali glavni oslonac za horizontalno spajanje promenljivih u širinu (svih kolona) i vertikalno slaganje različitih tipova ograničenja jedno ispod drugog (zahtevi servera, kapacitet baterija) u jedinstvenu formulaciju linearnog programiranja oblika $c^T x$ uz ograničenja $Ax \leq b$.

Izvršavanje i rezultat

Kada se ovako složena matrica prosledi **SciPy-jevom linprog solveru** (sa HiGHS algoritmom), sistem u jako brzo izračunava globalni optimum. Analizirajući izlazne podatke (**res.x**), tačno vidim isplativost cele operacije: u kom satu se baterija puni po niskoj tarifi, kada se prazni da pokrije energetske skokove servera, i koliki je minimalni ukupni trošak električne energije.

Jedna od ideja ovog problema jeste bila procena isplativosti uvođenja “zelene” energije u data centre. S obzirom da data centri koriste mnogo električne energije, ideja je bila uvesti solarne panele koji bi bili bolji za životnu sredinu i “olakšavali” elektroenergetsku mrežu, i potencijalno smanjili troškove korišćenja električne energije. Što direktno vodi ka energetskej efikasnosti i održivosti.

Zaključak

Kroz ove zadatke shvatio sam da optimizacija od mene ne traži samo da pišem petlje, već da fizičke i ekonomske zakone iz stvarnog sveta prevedem na jezik matrica. Prelazak sa jednostavnih uslova na matrične sisteme dao mi je sigurnost da mogu programski da rešavam složene inženjerske probleme sa desetinama zavisnih promenljivih. Shvatio sam da je optimizacija veoma važna oblast, koja može da dovodi do napretka ne samo softvera već i čitave industrije i zaštite životne sredine.

Repozitorijum (primeri u kodu): <https://github.com/marovichn/python-optimization>

Literatura: https://developers.google.com/optimization/cp/cp_solver

<https://or.stackexchange.com/questions/tagged/optimization>

<https://github.com/google/or-tools/tree/stable/examples/python>

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>

<https://www.geeksforgeeks.org/python/maximize-optimization-using-scipy/>

<https://www.w3schools.com/python/numpy/default.asp>